

KNITWEB · TECHNICAL WHITE PAPER

Decentralized Websites & the Role of Gither

Moving sha2.fail and sha256.fail off centralized hosting onto crypto-paid, peer-to-peer infrastructure — and where Gither completes the picture as Knitweb's ownership layer.

Knitweb

Version 1.0 · 2026.06

Knitweb · docs/gither

| Contents

01	Executive Summary	03
02	Background & Problem Statement	04
03	Findings — Decentralized Website Hosting	05
04	The Role of Gither	08
05	Conclusions & Recommendations	10
06	Appendix	11

01 · EXECUTIVE SUMMARY

| Executive Summary

A website depends on three centralized choke points: where the [code](#) lives, where the [running site](#) is hosted, and where the [domain](#) resolves. This paper shows how to remove the first two for `sha2.fail` and `sha256.fail` — replacing TransIP hosting with a decentralized compute network paid in crypto, and positioning Gither as the forge that removes the dependence on GitHub.

Key Takeaways

- For dynamic, server-side sites the practical decentralized host is [Akash Network](#) (Docker containers, paid in AKT/USDC); IPFS/Arweave fit only static content.
- Funding is self-custody and exchange-free: [NMR](#) → [Uniswap](#) → [bridge](#) → AKT/USDC, with the NMR sourced by unstaking from the Numerai crypto account ($\approx$ 2-week timelock).
- The domain stays at its normal registrar; only DNS records change — a reversible [CNAME cutover](#) to the Akash provider ingress.
- [Gither](#) is the missing third layer: a decentralized code forge that makes ownership and hosting of the source itself independent of GitHub.

QUESTIONS THIS DOCUMENT ANSWERS

Which decentralized host fits a dynamic site? How is it paid for with crypto we already hold? How do the `.fail` domains point at it without losing reversibility? And what role does Gither play in the same shift?

| Background & Problem Statement

The goal is a web presence that survives the withdrawal of any single provider. Today three independent vendors can each take a site offline.

Current State

`sha2.fail` and `sha256.fail` run on TransIP shared webhosting — a single operator that controls uptime, billing, and the execution environment. The source is held on a centralized forge (GitHub), and DNS resolves through the registrar. Each is a separate party with the unilateral power to suspend the service.

The Core Problem

Centralized hosting concentrates three distinct risks that a resilient site should decouple: platform lock-out, billing dependence on fiat rails, and operator-controlled uptime.

A site is only as sovereign as its *least* sovereign layer. Decentralizing hosting while leaving code ownership on a single platform simply moves the choke point one step upstream.

Dimensions of Success

Dimension	Centralized (TransIP + GitHub)	Target (decentralized)
Hosting authority	Single operator	Open provider marketplace
Payment rail	Fiat / card, KYC	Self-custody crypto
Code ownership	GitHub platform	Gither forge, Git mirrors optional
Domain control	Registrar (unchanged)	Registrar (unchanged), repointable

03 · FINDINGS

Findings — Decentralized Website Hosting

The right platform depends on whether a site is static or dynamic. Since `sha2.fail` / `sha256.fail` require a backend, the analysis centers on decentralized compute, not just decentralized storage.

Platform Comparison

Platform	Model	Pays in	Fits
Akash Network	Decentralized compute marketplace; runs Docker containers via an SDL manifest	AKT / USDC	Dynamic apps (recommended)
Flux	Decentralized PaaS; Docker apps across a node network	FLUX	Dynamic apps (alternative)
IPFS (Fleek / 4everland)	Content-addressed static hosting, pinned + gateway-served	Crypto via provider	Static sites only
Arweave	Permanent storage, one-time payment (permaweb)	AR	Static, write-once content

DECISION

Dynamic backend → [Akash](#). Stateful data (databases) does not belong on ephemeral providers without persistent storage and backups; keep canonical state on owned infrastructure.

Funding Path — NMR, Self-Custody

The hosting is paid from NMR already held in the Numerai crypto account. Unstaking returns the NMR to a self-custody wallet after an approximately [two-week timelock](#), so the unstake should be initiated first while the rest of the work proceeds in parallel. NMR is an ERC-20 token on Ethereum (contract `0x1776e1F26f98b1A5dF9cD347953a26dd3Cb46671`).

```
NMR (Numerai stake)
--[unstake, ~2 weeks]--> NMR in own wallet
  --[Uniswap]--> USDC (Ethereum)
    --[Squid / Axelar bridge]--> Osmosis (Cosmos)
      +--> swap a portion to AKT (Akash gas)
      +--[IBC]--> Akash wallet (Keplr) --> deploy
```

Only a fraction of the NMR is needed — a small container costs a few dollars per month — so the remainder can stay staked. A centralized exchange that lists both NMR and AKT is a shortcut, but it reintroduces KYC; the self-custody route above keeps the keys with the owner.

DNS Cutover

The `.fail` domains remain at the existing registrar; only the records change. The domains are declared in the Akash SDL under `expose → accept`, the provider issues TLS on its ingress, and DNS points at the provider hostname returned by `lease-status`.

Host	Type	Value
sha2.fail / www	CNAME	provider ingress hostname
sha256.fail / www	CNAME	provider ingress hostname

Lowering TTL to 300s a day ahead makes the switch fast and fully reversible: the TransIP records stay in place until the Akash deployment is verified, giving instant rollback.

| The Role of Gither

Decentralized hosting frees the *running* site. Gither frees the **source and its ownership** — the third choke point, and the one that hosting alone cannot solve.

What Gither Is

Gither is the Knitweb code forge: the trusted application layer for code ownership, change review, release gating, and repository collaboration. It is meant to replace GitHub and GitLab as the *authority* over code, while Git itself remains the open, interoperable object and transport layer underneath. GitHub and GitLab become optional mirrors, not the system of record.

Git stores code. Ethereum stores value. Gither stores ownership of knowledge.

— Gither, README

Why It Matters Here

The strategic risk of GitHub dependence is platform lockout. The weak answer is "move to GitLab," which still depends on servers, operators, and hosted uptime. Gither's target is a **serverless developer network** where code, changes, reviews, tests, releases, and CI receipts are verified through Knitweb records rather than trusted to one hosted service. It is positioned to be adopted first as a familiar peer-to-peer duplicate of GitHub, then to move toward Web3 and serverless development.

How the Layers Compose

Decentralizing the host without decentralizing the forge moves the choke point upstream rather than removing it. Gither closes that gap, and it does so within the wider Knitweb constellation:

- Gither records reviewed ownership and royalty policy — who owns what, and under which release gate.
- Knitweb / pulse records dependency relationships and real usage, so revenue can be attributed to the components actually used.
- Akash / IPFS / Arweave run and serve the resulting artifact.

Its incentive rule avoids commit-farming and AI-generated noise: code earns only when it is actually used. Ownership (Gither) and usage (pulse) are recorded separately, then reconciled.

Current Capabilities

Gither already provides repository discovery, task routing across Knitweb repos, repo-graph export, cross-repo test-plan output, Git repository snapshots, a Python code-discipline audit, versioned change notes in `.gither/changes`, and a local review gate for source state and Python quality.

CAVEAT — WHERE STATEFUL STATE LIVES

A forge is stateful (repositories, history, review records). That canonical state belongs on reliable, backed-up storage — not on an ephemeral compute provider. Decentralizing the forge is about authority and verifiability, with Git mirrors as durable backups, not about running the system of record on best-effort nodes.

| Conclusions & Recommendations

A sovereign web presence is achievable by decoupling all three layers — host, payment, and forge — each with a reversible migration path.

Core Conclusions

1. Akash is the right decentralized host for the dynamic `.fail` sites; IPFS/Arweave are reserved for any static assets.
2. The NMR-funded, self-custody payment path is viable and exchange-free, gated only by the two-week unstake timelock.
3. DNS cutover is low-risk and reversible, so the switch can be verified before TransIP is decommissioned.
4. Githr is the complementary forge layer: without it, decentralizing hosting leaves code ownership centralized on GitHub.

Recommended Next Steps

Run the funding clock and the build in parallel so they converge: initiate the NMR unstake now; during the timelock, containerize both sites and stage the Akash SDL and DNS records; on release of funds, swap, deploy, and flip DNS.

CALL TO ACTION

Start the NMR unstake on the Numerai crypto account today — it is the only step with a fixed two-week delay. Everything else can be prepared and tested while that clock runs.

| Appendix

A. References

- Gither — README, `github.com/Knitweb/gither`
- Knitweb — `docs/REPOSITORIES.md` (the repository constellation)
- Akash Network — SDL manifest and provider deployment documentation
- Companion artifact — `sha-fail-akash-hosting/` (`deploy.yaml`, `RUNBOOK.md`, Dockerfile templates)

B. Glossary

Akash SDL — Stack Definition Language; the manifest declaring images, ports, accepted domains, and resources for an Akash deployment.

AKT — the native token of the Akash chain; required for chain gas and deployment escrow.

NMR (Numeraire) — the ERC-20 token staked on Numerai; the funding source after unstaking.

IBC — Inter-Blockchain Communication; the Cosmos protocol used to move tokens to the Akash wallet.

Gither — Knitweb's decentralized code forge; the ownership, review, and release-gating layer above Git.

C. Status

Blocked on two inputs at time of writing: the source location and stack of `sha2.fail` / `sha256.fail` (needed to write the Dockerfiles), and the NMR unstake timelock. The hosting kit and this paper are the prepared, non-blocked deliverables.